

MACROFIT AI

Resumen del octavo bloque de trabajo

DIA 8 | Validacion JWT y primer endpoint protegido

Autor	Matias Gonzalez
Rama	feature/current-user
Commit	574eda8 - feat: add protected current user endpoint
Estado final	Endpoint protegido probado y rama sincronizada

Objetivo del documento

Documentar la validacion de tokens JWT recibidos, la identificacion del usuario autenticado y la creacion del primer endpoint protegido de MacroFit AI: GET /users/me.

1. Trabajo realizado

En este bloque se completo el ciclo de autenticacion. El backend no solo genera tokens durante el login, sino que ahora puede recibirlos, comprobarlos, identificar al usuario y proteger recursos privados.

- Integracion del login en develop y creacion de feature/current-user.
- Creacion de `get_user_by_id()` en el repository.
- Prueba de busqueda por ID existente e inexistente.
- Implementacion de `decode_access_token()`.
- Control de tokens manipulados, caducados o incompletos.
- Creacion de la dependencia `get_current_user()`.
- Extraccion de credenciales mediante HTTPBearer.
- Busqueda del usuario identificado por el claim sub.
- Creacion de `GET /users/me`.
- Registro del router de usuarios en `main.py`.
- Pruebas con token valido, sin token y token manipulado.

Flujo implementado

Authorization: Bearer JWT -> HTTPBearer -> decode_access_token -> user_id -> PostgreSQL -> UserResponse

2. Componentes desarrollados

Componente	Responsabilidad
get_user_by_id()	Busca en PostgreSQL al usuario identificado por su clave primaria.
decode_access_token()	Valida firma y expiracion, extrae sub y devuelve user_id.
HTTPBearer	Extrae el JWT de la cabecera Authorization.
get_current_user()	Coordina token, ID, repository y usuario autenticado.
GET /users/me	Devuelve los datos publicos del usuario autenticado.
UserResponse	Evita mostrar password_hash u otros campos sensibles.

Ubicacion de responsabilidades

- api/users.py: define el endpoint y devuelve la respuesta HTTP.
- core/security.py: valida y decodifica el token.
- core/dependencies.py: extrae el Bearer token y obtiene al usuario.
- repositories/user_repository.py: consulta PostgreSQL por ID.
- schemas/auth.py: UserResponse define la informacion publica.

3. Decodificacion segura del token

`decode_access_token()` utiliza PyJWT con la clave y el algoritmo configurados. La libreria verifica la firma y el claim exp. El valor sub se convierte a entero para consultar PostgreSQL.

Situacion	Resultado
Token valido con sub numerico	Devuelve el user_id como int.
Token manipulado	Captura InvalidTokenError y devuelve None.
Token caducado	Se considera invalido y devuelve None.
Token sin sub	Devuelve None.
sub no convertible a entero	Captura ValueError o TypeError y devuelve None.

- Un token valido para user_id 1 devolvio el entero 1.
- Al modificar el token, `decode_access_token()` devolvio None.
- El control de excepciones evita errores internos 500.
- La dependencia transforma los fallos de autenticacion en respuestas 401.

4. Dependencia get_current_user

La dependencia centraliza la autentificacion para que futuros endpoints no repitan la misma logica. FastAPI ejecuta automaticamente esta dependencia antes del endpoint protegido.

Paso	Accion
1	HTTPBearer obtiene las credenciales de Authorization.
2	credentials.credentials contiene solamente el JWT.
3	decode_access_token() devuelve user_id o None.
4	get_user_by_id() consulta la tabla users.
5	Si todo es valido, devuelve el objeto User.

- Token invalido o caducado: 401 Unauthorized.
- Usuario eliminado o inexistente: 401 Unauthorized.
- Token valido y usuario existente: devuelve User.
- La sesion de PostgreSQL se obtiene mediante Depends(get_db).

Reutilizacion futura

La misma dependencia podra proteger modificaciones de perfil, registros de peso, planes de comida, rutinas y cualquier operacion privada.

5. Pruebas realizadas

Prueba	Respuesta	Validacion
GET /users/me con token valido	200 OK	Devuelve el usuario autenticado.
GET /users/me sin token	401 Unauthorized	HTTPBearer rechaza la peticion.
GET /users/me con token manipulado	401 Unauthorized	La firma no es valida.
get_user_by_id(1)	Objeto User	Encuentra el usuario real.
get_user_by_id(999999)	None	Gestiona ID inexistente.

- Swagger mostro un candado en GET /users/me.
- El boton Authorize permitio configurar el Bearer token.
- UserResponse devolvio id, name, email, idioma, estado y fecha.
- No se expusieron password ni password_hash.
- Los logs de Uvicorn confirmaron respuestas 200 y 401.

6. Aprendizajes y estado final

Concepto	Aprendizaje
Dependencias	Depends permite ejecutar logica reutilizable antes de un endpoint.
HTTPBearer	Extrae el esquema Bearer y el token de Authorization.
JWT sub	Identifica al usuario propietario del token.
try-except	Controla errores esperados sin devolver 500.
Repository	Realiza la busqueda por ID y devuelve User o None.
Endpoint protegido	Solo se ejecuta si la dependencia autentica la peticion.

- Commit 574eda8 creado: feat: add protected current user endpoint.
- Rama feature/current-user subida a GitHub.
- Working tree limpio al finalizar.
- Registro, login y recuperacion del usuario autenticado ya funcionan como un flujo completo.

Proximos pasos

- Actualizar el backlog y subir este informe.
- Integrar feature/current-user en develop.
- Crear el flujo de completar perfil.
- Permitir consultar y modificar el perfil autenticado.
- Añadir pruebas automatizadas de autenticacion.
- Introducir Alembic antes de ampliar el esquema de base de datos.

HITO: MacroFit AI ya identifica al usuario mediante JWT y protege recursos privados.